APLICATIVO PARA DEPURACIÓN Y CONVERSIÓN DE GRAMATICA LIBRES DE CONTEXTO A FNC

PRESENTADO POR:
DIEGO ANDRÉS ALVAREZ LIZARAZO
JUAN DAVID QUINTERO MELO
DIEGO ARMANDO MARQUEZ RIGAUD

TEORÍA DE LA COMPUTACIÓN

DOCENTE:
CARMEN JANETH PARADA

UNIVERSIDAD FRANCISCO DE PAULA SANTANDER
FACULTAD DE INGENIERÍA
CÚCUTA, NORTE DE SANTANDER
30 DE SEPTIEMBRE DE 2026

LISTA DE CONTENIDO

INTRODUCCIÓN..... 4
1. PRESENTACIÓN GENERAL DEL PROYECTO..... 5
1.1 TITULO DE PROYECTO..... 5
1.2 PLANTEAMIENTO DEL PROBLEMA..... 5
1.3 OBJETIVOS..... 8
1.3.1 OBJETIVO GENERAL..... 8
1.3.2 OBJETIVOS ESPECÍFICOS..... 8
2. MARCO TEÓRICO..... 9
3. REQUERIMIENTOS..... 13
3.1 REQUERIMIENTOS FUNCIONALES..... 14
3.2 REQUERIMIENTOS NO FUNCIONALES..... 17
4. ANÁLISIS Y DISEÑO..... 18
5. ALGORITMOS IMPLEMENTADOS..... 21
6. IMPLEMENTACIÓN..... 26
7. PRUEBAS..... 31
8. RESULTADOS..... 50
9. CONCLUSIONES..... 54
10. REFERENCIAS BIBLIOGRÁFICAS..... 56

LISTA DE TABLAS

Tabla 1 Requerimientos Funcionales..... 13
Tabla 2 Requerimientos no funcionales..... 16
Tabla 3 Resultados Pruebas..... 42

LISTA DE FIGURAS

Figura 1. Diagrama general Casos de Uso..... 17
Figura 2. Diagrama de Clases..... 18
Figura 3. Diagrama de Actividades..... 19
Figura 4. Validación inicial de la gramática..... 31
Figura 5. Eliminación de variables inútiles (símbolos no declarados)..... 33
Figura 6. Eliminación de producciones nulas con ciclo emergente a mitad de proceso..... 35
Figura 7. Eliminación de producciones nulas con ciclo emergente a mitad de proceso..... 36
Figura 8. Eliminación de producciones nulas con ciclo emergente a mitad de proceso..... 37
Figura 9. Eliminación de producciones unitarias en cadena (un nivel a la vez)..... 39
Figura 10. Conversión a Forma Normal de Chomsky..... 42
Figura 11. Integración completa..... 46
Figura 12. Integración completa..... 47
Figura 13. Integración completa..... 48

INTRODUCCIÓN

Las gramáticas libres de contexto son la base de compiladores, intérpretes y analizadores sintácticos. Sin embargo, casi nunca están en la forma adecuada para procesarse de manera eficiente. Algoritmos como el CYK exigen la Forma Normal de Chomsky (FNC), y llegar a ella requiere depurar la gramática, eliminando variables inútiles, inalcanzables, producciones nulas y unitarias, para luego binarizar las producciones. Este proyecto surge en la asignatura Teoría de la Computación, donde ese procedimiento se resuelve a mano.

Hacerlo manualmente es propenso al error. Una sola eliminación puede afectar variables que ya se creían resueltas, y el estudiante solo descubre el fallo al final, cuando es difícil ubicar el paso donde se originó. Esto genera calificaciones que no reflejan la comprensión real y debilita el razonamiento algorítmico que el ejercicio busca consolidar. De ahí surge la necesidad de una herramienta que reproduzca el proceso paso a paso y con retroalimentación inmediata. No reemplaza el razonamiento del estudiante, sino que le da un punto de referencia confiable para contrastar su trabajo y detectar en qué momento se desvió.

Por ello, el objetivo general es desarrollar un aplicativo web para la depuración y conversión automática de gramáticas libres de contexto a su Forma Normal de Chomsky. Para lograrlo se implementaron la validación estructural de la gramática, la eliminación de variables inútiles e inalcanzables, la eliminación de producciones nulas y unitarias resolviendo una variable a la vez, la binarización de producciones y una numeración progresiva de los estados intermedios (Σ_0, Σ_1, Σ_2…).

Para su desarrollo se levantaron los requerimientos, se elaboraron los modelos de diseño y se programaron los algoritmos en Python con Flask, con una interfaz en React y Tailwind CSS desplegada en Vercel, de modo que el aplicativo se use desde el navegador. Los resultados se validaron con casos de prueba comparados contra el cálculo manual. El sistema procesa cualquier gramática ingresada, ofrece modo manual y automático y genera ejercicios de práctica, aunque adopta una FNC que admite cualquier combinación de dos símbolos por producción, según la forma de trabajo de la asignatura.

El documento se organiza en diez capítulos: presentación del proyecto, marco teórico, requerimientos, análisis y diseño, algoritmos, implementación, pruebas, resultados, conclusiones y referencias bibliográficas.

1. PRESENTACIÓN GENERAL DEL PROYECTO

1.1 TITULO DE PROYECTO

Aplicativo Para Depuración Y Conversión De Gramática Libres De Contexto A Fnc.

1.2 PLANTEAMIENTO DEL PROBLEMA

La formalización de los lenguajes mediante gramáticas libres de contexto es uno de los pilares sobre los que se sostiene buena parte de la computación moderna: todo compilador, todo intérprete y todo analizador sintáctico desde los que procesan un lenguaje de programación hasta los que se usan en tareas de procesamiento de lenguaje natural depende de que exista una gramática bien definida detrás de la estructura que se quiere reconocer. Sin embargo, una gramática libre de contexto tal como se define inicialmente casi nunca está en la forma más adecuada para ser procesada de manera eficiente. Algoritmos de análisis sintáctico ampliamente utilizados en la práctica, como el algoritmo CYK (Cocke-Younger-Kasami), solo pueden aplicarse sobre gramáticas que cumplen una estructura muy específica: la Forma Normal de Chomsky, en la que toda producción se reduce a, como máximo, dos símbolos. Autores de referencia en la teoría de lenguajes formales, como Hopcroft y Ullman, señalan que este tipo de normalización no es un simple formalismo académico, sino una condición necesaria para que los algoritmos de análisis sintáctico funcionen con una complejidad computacional predecible y manejable. En otras palabras: sin depuración ni normalización, una gramática es, en la práctica, mucho más difícil de aprovechar computacionalmente.

Ese proceso de depuración y conversión identificar y eliminar variables inútiles, variables inalcanzables, producciones nulas y producciones unitarias, para luego reestructurar lo que queda hasta que cada producción tenga máximo dos símbolos es también, en el ámbito académico, uno de los ejercicios formativos más exigentes para quien se está formando en fundamentos de lenguajes formales. El escenario específico donde se sitúa este proyecto es precisamente ese: el de un estudiante de ingeniería que, durante un periodo académico, debe resolver a mano, sobre el papel, este tipo de transformaciones sobre gramáticas de tamaño y complejidad variables.

Ahí aparece una brecha clara entre el estado ideal y el estado real. Idealmente, el estudiante debería poder ejecutar cada fase del proceso con precisión, llevar un control ordenado de cada gramática intermedia, y llegar a una Forma Normal de Chomsky correcta y verificable. En la práctica, sin embargo, es común que el proceso se pierda a mitad de camino: producciones que deberían eliminarse permanecen escondidas entre decenas de otras, el orden en que se resuelven las nulas y las unitarias no siempre es evidente, y el resultado final aunque parezca correcto a primera vista puede seguir conteniendo una producción nula residual o una unitaria mal resuelta que invalida por completo la conversión.

Varias causas explican por qué este proceso es tan propenso al error cuando se ejecuta manualmente. En primer lugar, la naturaleza combinatoria del proceso: eliminar una sola producción nula puede requerir generar múltiples variantes de varias producciones distintas a la vez, y ese número de variantes crece rápidamente conforme aumenta el número de símbolos anulables presentes en una misma producción. En segundo lugar, la interdependencia entre variables: la eliminación de una producción unitaria o nula en una variable puede modificar, de forma indirecta, las producciones de otra variable que ya se creía resuelta, generando así un efecto en cadena difícil de anticipar sin una revisión constante del estado completo de la

gramática. Y, en tercer lugar, la ausencia de retroalimentación inmediata: cuando el ejercicio se resuelve completamente a mano, el estudiante solo puede confirmar si su proceso fue correcto hasta llegar al resultado final, y para entonces ya es difícil identificar en qué paso intermedio se originó el error.

Las consecuencias de esto no son menores. Un error de organización no de comprensión conceptual puede llevar a que un estudiante entregue una gramática que, pese a haber aplicado correctamente la teoría en su cabeza, no cumple realmente con la Forma Normal de Chomsky. Esto se traduce en calificaciones que no reflejan el verdadero nivel de comprensión del estudiante, en tiempo invertido que se pierde persiguiendo un error sin saber dónde se originó, y, más preocupante aún, en una consolidación débil del razonamiento algorítmico detrás del proceso, precisamente el aprendizaje que este tipo de ejercicio busca dejar.

Frente a esta situación, resulta pertinente contar con una herramienta que reproduzca fielmente el mismo procedimiento que se exige realizar a mano con la misma notación, el mismo orden de fases y el mismo tratamiento de los casos especiales que suelen generar confusión, como una producción nula que reaparece a mitad de proceso, pero que lo ejecute de forma automática y verificable en cada uno de sus pasos. Una solución de este tipo no reemplaza el ejercicio de razonar la gramática; le da al estudiante, y a cualquier persona que esté aprendiendo este tema, un punto de referencia inmediato y confiable contra el cual contrastar su propio trabajo, detectar en qué paso específico se desvió, y reforzar con mayor seguridad el dominio del proceso.

A partir de lo anterior, este proyecto se propone responder la siguiente pregunta: ¿De qué manera la ausencia de una herramienta que reproduzca, paso a paso y con retroalimentación inmediata, el proceso de depuración y conversión de una gramática libre de contexto a su Forma

Normal de Chomsky, dificulta que un estudiante pueda verificar y consolidar correctamente el dominio de ese proceso algorítmico?

1.3 OBJETIVOS

1.3.1 OBJETIVO GENERAL

Desarrollar un aplicativo web para la depuración y conversión automática de Gramáticas Libres de Contexto a su Forma Normal de Chomsky, como herramienta de verificación paso a paso para el dominio de dicho proceso algorítmico, en el contexto del aprendizaje de los fundamentos de los lenguajes formales.

1.3.2 OBJETIVOS ESPECÍFICOS

- Implementar el módulo de registro y validación estructural de la gramática libre de contexto, como base para el inicio del proceso de depuración.
- Programar los algoritmos de eliminación de variables inútiles e inalcanzables, para la reducción de la gramática a sus componentes generadores y alcanzables desde el símbolo inicial.
- Desarrollar el algoritmo de eliminación de producciones nulas y unitarias mediante resolución individual por variable, para el control de los casos de dependencia cíclica que surgen entre ellas.
- Construir el algoritmo de conversión a Forma Normal de Chomsky mediante la binarización de las producciones, para la obtención de una gramática equivalente apta para algoritmos de análisis sintáctico.

- Incorporar un sistema de numeración progresiva de los estados intermedios de la gramática, para la identificación del punto exacto del proceso en que se origina cada cambio.

2. MARCO TEÓRICO

Lenguaje formal, alfabeto, cadena y cadena vacía

Un alfabeto es un conjunto finito y no vacío de símbolos, generalmente denotado con la letra griega Σ (sigma) en la literatura clásica de lenguajes formales. A partir de un alfabeto se pueden construir cadenas, es decir, secuencias finitas de símbolos tomados de ese alfabeto y concatenados entre sí. La cadena de longitud cero aquella que no contiene ningún símbolo recibe el nombre de cadena vacía, y se representa habitualmente con el símbolo λ (lambda) o ε (épsilon); en este proyecto se adoptó la notación λ para mantener coherencia con la forma en que se trabaja el concepto en el aula.

Un lenguaje formal es, en su definición más simple, cualquier subconjunto de todas las cadenas posibles que se pueden construir sobre un alfabeto dado. Esta noción, aunque abstracta, es la base sobre la que se sostiene toda la teoría de compiladores y de procesamiento de lenguajes: un lenguaje de programación, por ejemplo, no es más que un lenguaje formal cuyas cadenas válidas son los programas sintácticamente correctos que ese lenguaje admite.

Gramática formal y Gramática Libre de Contexto

Una gramática formal es un mecanismo que permite describir, mediante un conjunto finito de reglas, un lenguaje formal que en general contiene un número infinito de cadenas. En

lugar de enumerar una por una las cadenas que pertenecen al lenguaje, la gramática establece cómo construirlas a partir de una serie de sustituciones.

Una Gramática Libre de Contexto (GLC) es un tipo particular de gramática formal en la que cada regla de sustitución reemplaza una única variable por una secuencia de símbolos, sin importar el contexto en el que esa variable aparezca. Formalmente, se define como una tupla G = (V, T, S, P), donde:

- **V** es el conjunto de variables o no terminales: símbolos que representan estructuras sintácticas abstractas y que pueden seguir sustituyéndose.
- **T** es el conjunto de terminales: los símbolos que efectivamente componen las cadenas del lenguaje y que ya no pueden sustituirse más.
- **S** es el símbolo inicial, una variable distinguida de V a partir de la cual comienza cualquier derivación.
- **P** es el conjunto de producciones: reglas de la forma $A \rightarrow \alpha$, donde A es una variable y α es una cadena compuesta por cualquier combinación de variables y terminales, incluyendo la posibilidad de que α sea la cadena vacía.

En este proyecto, siguiendo la convención de trabajo adoptada, el conjunto de producciones se identifica con la letra Σ, y cada estado intermedio de la gramática a lo largo del proceso de depuración se numera de forma progresiva (Σ_0, Σ_1, Σ_2…), de manera que cada transformación quede asociada a un estado claramente identificable.

Derivación y gramática equivalente

Una derivación es el proceso de aplicar sucesivamente producciones a partir del símbolo inicial hasta obtener una cadena compuesta únicamente por terminales. El lenguaje generado por

una gramática es, precisamente, el conjunto de todas las cadenas de terminales que pueden obtenerse mediante alguna derivación válida a partir de S.

Se dice que dos gramáticas son gramáticas equivalentes cuando, a pesar de tener conjuntos de variables o producciones distintas, generan exactamente el mismo lenguaje. Todo el proceso de depuración y conversión a Forma Normal de Chomsky que implementa este aplicativo se sustenta en esta noción: en cada transformación eliminar una variable inútil, resolver una producción nula, binarizar una producción larga la gramática cambia en su forma, pero el lenguaje que genera debe permanecer intacto. La única excepción formalmente aceptada a este principio es el tratamiento especial de la cadena vacía, que se explica en la sección FNC.

Variable anulable, producción nula y producción unitaria

Una producción nula es una regla de la forma $A \rightarrow \lambda$, es decir, una producción cuyo lado derecho es la cadena vacía. Una variable que posee, directa o indirectamente, una manera de derivar en la cadena vacía se denomina variable anulable. La anulabilidad puede ser directa (cuando la propia variable tiene una producción nula explícita) o indirecta (cuando una variable deriva, a través de una o más producciones intermedias, en otra variable que sí es anulable). Esta distinción resulta especialmente relevante en la práctica: como se evidenció durante el desarrollo del proyecto, una variable puede volverse anulable como efecto colateral de la eliminación de la nula de otra variable distinta, sin que esa anulabilidad fuera visible en la gramática original.

Una producción unitaria es una regla de la forma $A \rightarrow B$, donde tanto el lado izquierdo como el lado derecho son una única variable. Este tipo de producción no aporta información estructural nueva simplemente redirige de una variable a otra y por ello debe eliminarse, sustituyendo la producción unitaria por las producciones no unitarias de la variable de destino.

Al igual que ocurre con la anulabilidad, la resolución de una producción unitaria puede generar, de forma indirecta, nuevas producciones nulas o unitarias en otras variables que dependían de ella, razón por la cual ambos procesos deben tratarse de forma iterativa y no como un único paso aislado.

Variable generadora, no generadora, alcanzable, inalcanzable e inútil

Una variable es generadora cuando, mediante alguna secuencia finita de derivaciones, es capaz de producir una cadena compuesta únicamente por símbolos terminales. Por el contrario, una variable es no generadora cuando ninguna de sus producciones ni directa ni indirectamente conduce jamás a una cadena de solo terminales, habitualmente porque queda atrapada en una dependencia circular con otras variables igualmente no generadoras.

Una variable es alcanzable cuando existe al menos una secuencia de derivaciones, comenzando en el símbolo inicial, que permite llegar a ella. Cuando no existe tal secuencia, la variable se considera inalcanzable: aunque esté correctamente definida y tenga producciones válidas, nunca podrá participar en la generación de ninguna cadena del lenguaje, sencillamente porque no hay forma de llegar hasta ella desde S.

El concepto de variable inútil agrupa ambos casos: toda variable no generadora, toda variable inalcanzable, y en un sentido más amplio, adoptado en este proyecto todo símbolo que se emplee en alguna producción sin haber sido formalmente declarado como variable o terminal, dado que un símbolo en esa condición nunca podrá generar una cadena de terminales por sí mismo. Eliminar las variables inútiles, junto con toda producción que las contenga, es siempre el primer paso del proceso de depuración, ya que su presencia puede ocultar o alterar el resultado de las fases posteriores.

Forma Normal de Chomsky

Una Gramática Libre de Contexto se encuentra en Forma Normal de Chomsky (FNC) cuando todas sus producciones tienen, como máximo, dos símbolos en el lado derecho. Bajo la definición clásica más estricta, únicamente se admiten producciones de la forma $A \rightarrow BC$ (dos variables) o $A \rightarrow a$ (un único terminal); no obstante, en la forma de trabajo adoptada para este proyecto se acepta cualquier combinación de dos símbolos terminal-terminal, terminal-variable, variable-terminal o variable-variable, siempre que la producción no supere los dos símbolos de longitud.

La única excepción permitida a esta estructura es la producción $S \rightarrow \lambda$, aceptada únicamente cuando S es el símbolo inicial y la cadena vacía pertenece efectivamente al lenguaje original; en cualquier otra variable, una producción nula constituye un error de conversión. Alcanzar esta forma normal requiere, cuando una producción supera los dos símbolos, introducir variables auxiliares nuevas que descompongan la producción original en una cadena de producciones binarias equivalentes, hasta que ninguna quede fuera de la estructura permitida.

La importancia práctica de la Forma Normal de Chomsky radica en que gran parte de los algoritmos de análisis sintáctico eficientes como el algoritmo CYK o FNG exigen que la gramática de entrada esté normalizada de esta manera para poder operar con una complejidad computacional predecible. Por esta razón, todo el proceso de depuración descrito en las secciones anteriores no es un fin en sí mismo, sino el paso previo indispensable para alcanzar esta forma normal sin alterar el lenguaje que la gramática original estaba destinada a representar.

3. REQUERIMIENTOS

3.1 REQUERIMIENTOS FUNCIONALES

Tabla 1

Requerimientos funcionales

CÓDIGO	REQUERIMIENTO
RF01	El sistema deberá permitir registrar una nueva Gramática Libre de Contexto, ingresando su conjunto de variables, su conjunto de terminales, su símbolo inicial y sus reglas de producción.
RF02	El sistema deberá validar la estructura básica de la gramática ingresada (existencia de al menos una variable, al menos un terminal, y un símbolo inicial que pertenezca al conjunto de variables), impidiendo el inicio del proceso de depuración mientras existan errores de este tipo.
RF03	El sistema deberá identificar las variables inútiles, entendidas tanto como variables declaradas sin producciones propias, como símbolos usados en alguna producción sin haber sido declarados como variable ni como terminal.
RF04	El sistema deberá eliminar las variables inútiles identificadas, junto con toda producción que las contenga en cualquier otra variable de la gramática.
RF05	El sistema deberá identificar y eliminar las variables inalcanzables desde el símbolo inicial.
RF06	El sistema deberá identificar las variables anulables y eliminar sus producciones nulas, resolviendo cada variable de forma independiente y registrando cada resolución como una transformación separada.
RF07	El sistema deberá detectar cuándo la eliminación de una producción nula genera,

	de forma indirecta, una nueva producción nula en otra variable incluyendo los casos en que dicha variable ya había sido procesada previamente, y encolarla para su resolución en un paso posterior.
RF08	El sistema deberá identificar y eliminar las producciones unitarias, sustituyendo cada variable de origen únicamente con las producciones vigentes de su variable de destino directa, sin incorporar de forma prematura las producciones de destinos indirectos aún no resueltos.
RF09	El sistema deberá actualizar automáticamente una variable de origen cuando alguna de las variables de las que depende sea modificada en un paso posterior, propagando dicho cambio hasta que ninguna variable conserve producciones unitarias pendientes.
RF10	El sistema deberá detectar los casos en que la eliminación de producciones nulas y unitarias entra en un ciclo de dependencia mutua, y resolverlo alternando automáticamente entre ambas fases hasta que el proceso converja.
RF11	El sistema deberá convertir la gramática depurada a su Forma Normal de Chomsky, descomponiendo toda producción de longitud mayor a dos símbolos mediante la creación de variables auxiliares nuevas.
RF12	El sistema deberá nombrar las variables auxiliares de forma secuencial (X1, X2, X3…), evitando cualquier nombre ya existente en la gramática, y asignándolas en el mismo orden en que se requieran durante la binarización de cada producción.
RF13	El sistema deberá validar automáticamente que la gramática resultante cumpla con la Forma Normal de Chomsky, reportando cualquier producción que no cumpla la estructura exigida.
RF14	El sistema deberá numerar cada estado

	intermedio de la gramática (Σ_0, Σ_1, Σ_2…), incrementando el número únicamente cuando una fase produzca un cambio real sobre la gramática.
RF15	El sistema deberá mostrar, para cada transformación realizada, la gramática antes y después del paso, los elementos identificados, y las producciones eliminadas o agregadas.
RF16	El sistema deberá ofrecer un modo de ejecución manual, en el que el usuario decide qué fase ejecutar en cada momento, y un modo de ejecución automático, que resuelve la totalidad del proceso en una sola acción.
RF17	El sistema deberá presentar el proceso de conversión a Forma Normal de Chomsky diferenciando visualmente las producciones que quedan dentro de cada variable de las nuevas variables auxiliares creadas.
RF18	El sistema deberá permitir generar una gramática aleatoria como ejercicio de práctica, con el fin de que el usuario pueda resolverla por su cuenta y comparar su resultado contra el que produce el sistema.
RF19	El sistema deberá permitir reiniciar el proceso, descartando la gramática y el historial actuales para ingresar una nueva gramática.

Nota. Elaboración Propia

3.2 REQUERIMIENTOS NO FUNCIONALES

Tabla 2

Requerimientos no funcionales

CÓDIGO	REQUERIMIENTO
RNF01	El sistema deberá contar con una interfaz web comprensible, que reproduzca visualmente la notación y el orden de trabajo utilizados en el desarrollo manual del proceso.
RFN02	Los mensajes de validación y de error deberán ser claros y específicos, indicando el elemento exacto que los origina.
RFN03	El código fuente deberá estar organizado por módulos según su responsabilidad (representación de la gramática, validación, depuración, conversión a Forma Normal de Chomsky, generación de ejercicios), y debidamente comentado.
RFN04	El sistema deberá procesar cualquier gramática libre de contexto que el usuario ingrese, sin limitarse a casos predefinidos.
RFN05	Las transformaciones realizadas deberán conservar el lenguaje generado por la gramática original.
RFN06	El sistema deberá evitar la duplicación de producciones dentro de una misma variable a lo largo de todo el proceso.
RFN07	El resultado del proceso deberá ser reproducible: una misma gramática de entrada deberá producir, en ejecuciones distintas, exactamente la misma secuencia de transformaciones y el mismo resultado final.
RFN08	El sistema deberá registrar la totalidad de los pasos efectuados durante el proceso, sin omitir ninguna transformación intermedia.
RFN09	El aplicativo deberá ser accesible desde un

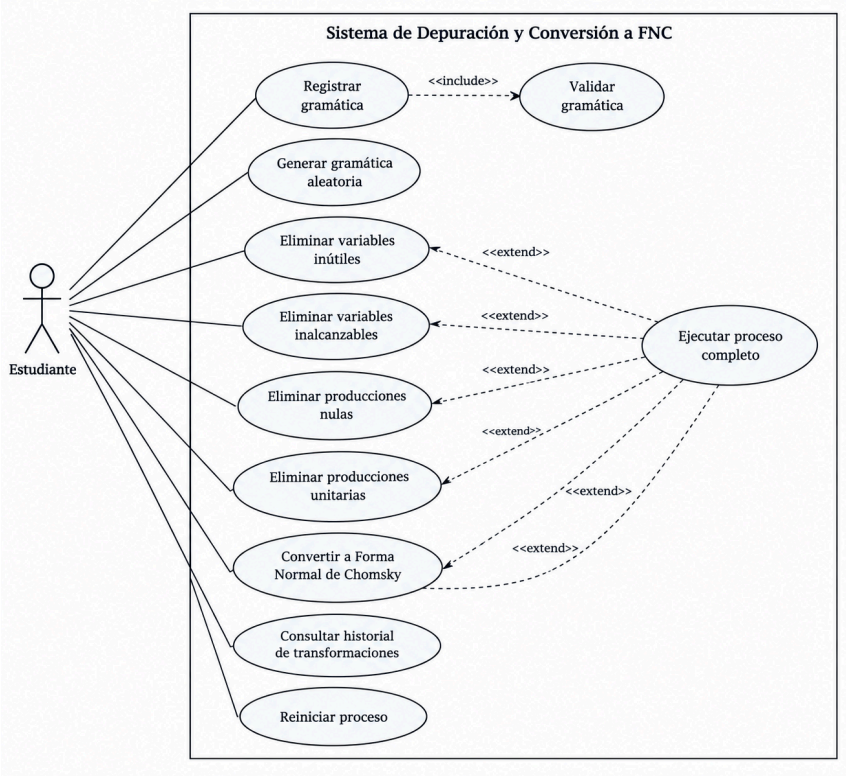
	navegador web, sin requerir instalación local por parte del usuario.
RFN10	El backend del sistema, encargado de la lógica de depuración y conversión, deberá desplegarse como un servicio independiente consumido por la interfaz mediante peticiones HTTP.
RFN11	El generador de gramáticas de práctica deberá garantizar que la gramática producida no degenere en un resultado vacío tras el proceso de depuración.

Nota. Elaboración Propia

4. ANÁLISIS Y DISEÑO

CASOS DE USO

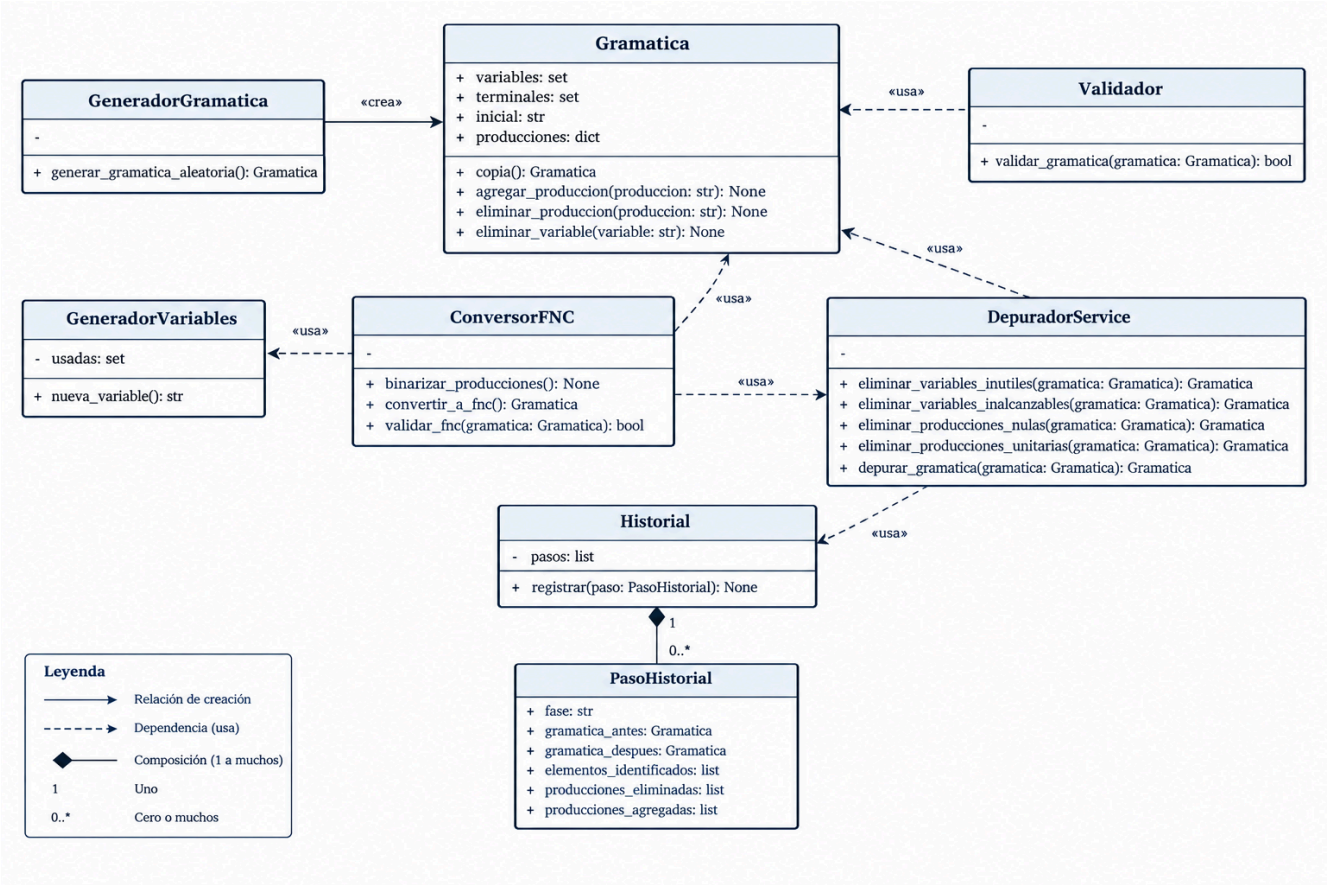
Figura 1. Diagrama general Casos de Uso



Nota. Elaborado con IA

DIAGRAMA DE CLASES

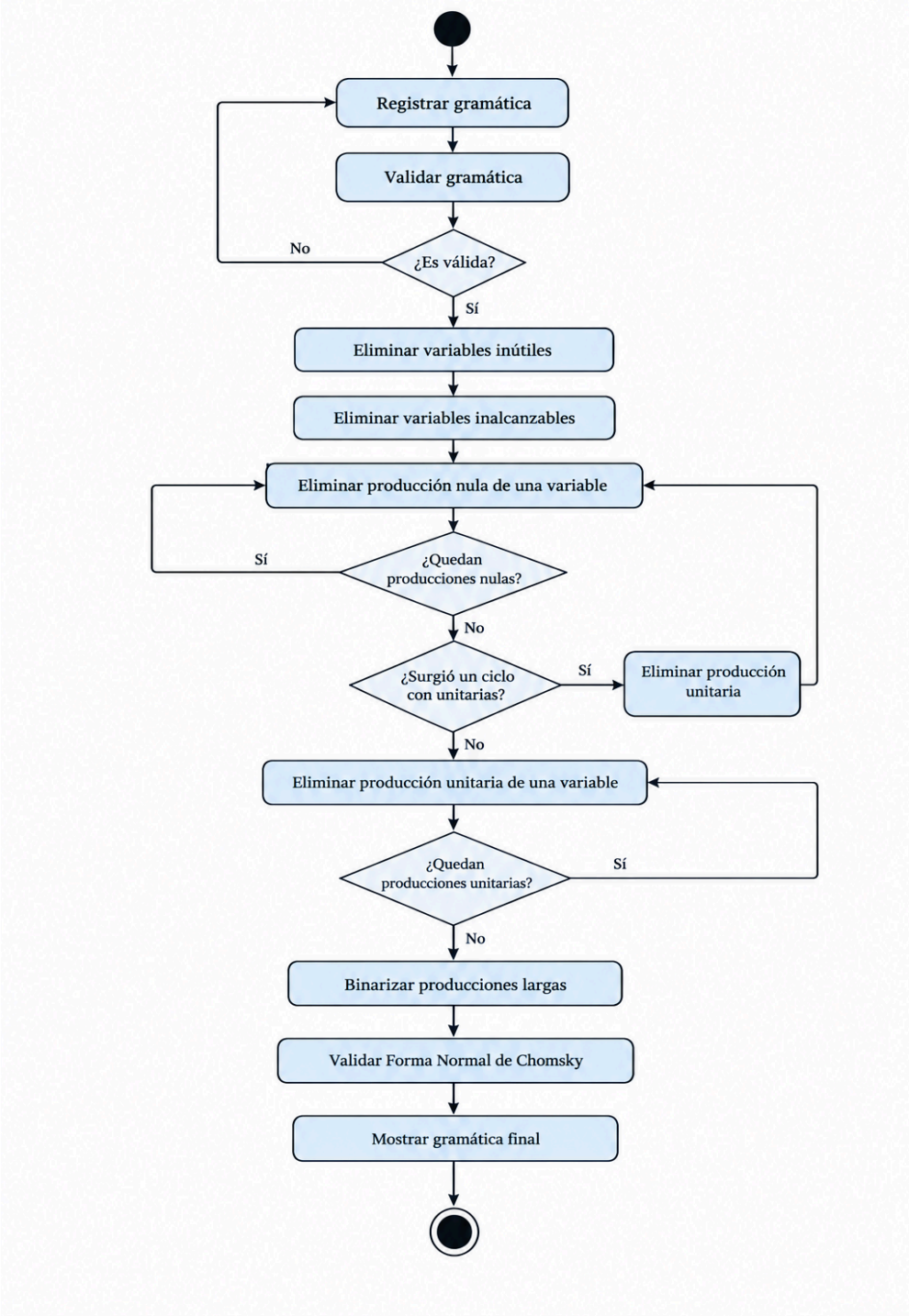
Figura 2. Diagrama de Clases



Nota. Elaborado con IA

DIAGRAMA DE ACTIVIDADES

Figura 3. Diagrama de Actividades



Nota. Elaborado con IA

5. ALGORITMOS IMPLEMENTADOS

Identificación de variables generadoras

Este algoritmo determina qué variables son capaces de derivar, en algún número finito de pasos, una cadena compuesta únicamente por símbolos terminales. Es la base sobre la que se construye la eliminación de variables inútiles.

El algoritmo funciona mediante un cálculo de punto fijo:

- Se inicia con un conjunto de variables generadoras vacío.
- Se recorre cada variable de la gramática: si alguna de sus producciones está compuesta únicamente por terminales, o es la producción nula, esa variable se marca como generadora.
- Se repite el recorrido: en cada nueva pasada, se marca como generadora toda variable que tenga una producción donde todos los símbolos terminales o variables ya estén marcados como generadores.
- El proceso se detiene cuando una pasada completa no logra marcar ninguna variable nueva.

Toda variable que, al finalizar, no quede marcada como generadora, se considera no generadora.

Eliminación de variables inútiles

Se define como variable inútil no solo aquella que no es generadora, sino también todo símbolo empleado en alguna producción sin haber sido declarado formalmente como variable ni

como terminal un caso que, en la práctica, resulta igual de frecuente que el anterior. El algoritmo procede así:

- Se recorren todas las producciones de la gramática en busca de símbolos que no pertenezcan ni al conjunto de variables ni al de terminales.
- Cada símbolo no declarado se incorpora temporalmente como una variable sin producciones propias, de modo que el cálculo de variables generadoras la identifica automáticamente como no generadora.
- Se ejecuta el algoritmo de identificación de variables generadoras sobre la gramática resultante.
- Toda variable que no quede marcada como generadora incluyendo las incorporadas en el paso 2 se elimina, junto con cualquier producción, de cualquier otra variable, que la contenga.

Identificación de variables alcanzables y eliminación de variables inalcanzables

Este algoritmo determina qué variables pueden llegar a aparecer en alguna derivación que comience en el símbolo inicial, mediante un recorrido en anchura o profundidad sobre el grafo de dependencias de la gramática:

- Se inicia un conjunto de variables alcanzables que contiene únicamente al símbolo inicial.
- Se examinan las producciones de cada variable ya marcada como alcanzable; toda variable que aparezca en el lado derecho de alguna de esas producciones se agrega al conjunto y queda pendiente de examinar.
- El proceso continúa hasta que no queden variables pendientes de examinar.

Toda variable que no forme parte del conjunto resultante se elimina, junto con las producciones que la contengan. Esta fase se ejecuta siempre después de la eliminación de variables inútiles, ya que la desaparición de una producción durante esa fase puede romper el único camino que conectaba al símbolo inicial con otra variable, volviéndola inalcanzable de forma indirecta.

Eliminación de producciones nulas

Este es uno de los algoritmos de mayor complejidad del proyecto, y su diseño responde directamente a la forma en que el proceso se trabaja de manera manual: una variable anulable a la vez, y no todas simultáneamente.

Identificación de variables anulables. De forma análoga al cálculo de variables generadoras, se determina mediante punto fijo qué variables pueden derivar, directa o indirectamente, en la cadena vacía: una variable es anulable si tiene una producción nula explícita, o si tiene una producción compuesta enteramente por símbolos ya identificados como anulables.

Eliminación variable por variable. Para cada variable con una producción nula vigente:

- Se elimina la producción nula propia de esa variable.
- Se recorren las producciones de toda la gramática en busca de ocurrencias de esa variable. Por cada producción que la contenga, se generan todas las variantes posibles quitando subconjuntos de sus ocurrencias, evitando duplicados y preservando siempre la producción original.

- Si, como consecuencia de este proceso, alguna otra variable queda con una producción vacía nueva es decir, se vuelve anulable de forma indirecta, dicha variable se agrega a la cola de procesamiento para ser resuelta en un paso posterior, nunca dentro del mismo paso.

Detección de ciclos emergentes. El aspecto más delicado del algoritmo es que una dependencia cíclica entre dos variables no siempre es visible desde el inicio del proceso: puede surgir a mitad de camino, como efecto colateral de una eliminación anterior. Por esta razón, el algoritmo lleva un registro de cuántas veces se ha procesado cada variable; si una variable que ya fue resuelta necesita resolverse nuevamente, esto se interpreta como la señal inequívoca de un ciclo de dependencia mutua. En ese momento, el proceso de eliminación de nulas se pausa, se ejecuta una pasada de eliminación de producciones unitarias sobre el estado actual lo que fusiona las producciones de las variables implicadas y rompe la dependencia circular, y a continuación se reanuda la eliminación de nulas sobre la gramática resultante.

Eliminación de producciones unitarias

Al igual que con las producciones nulas, este algoritmo resuelve una variable de origen a la vez, y no mediante el cierre transitivo completo en un único paso. Esta decisión de diseño es intencional: sustituir una variable por el contenido íntegro de toda su cadena de dependencias unitarias en un solo movimiento mezcla niveles que, en el proceso manual, se resuelven por separado.

- Para cada variable de origen que tenga al menos una producción unitaria vigente, se identifican todas sus variables de destino directas.

- Se sustituye cada producción unitaria por las producciones actuales de la variable de destino correspondiente, excluyendo explícitamente cualquier producción unitaria que el destino aún conserve esa se resolverá en el turno propio del destino, no en el de origen. Si varias producciones unitarias parten de la misma variable de origen, todas se resuelven juntas en un único paso.
- Se registra qué variable sustituyó a través de cuál destino, de forma que, si ese destino se modifica en un paso posterior por ejemplo, al resolver su propia unitaria, la variable de origen se actualice automáticamente, heredando el contenido nuevo sin necesidad de repetir la sustitución completa.
- El proceso continúa hasta que ninguna variable conserva producciones unitarias pendientes.

Este mecanismo de actualización por dependencia es lo que permite que el algoritmo llegue a un resultado completo y correcto incluso en cadenas de varios niveles (por ejemplo, A → B → C), sin perder alcanzabilidad, y sin necesidad de calcular de antemano el cierre transitivo completo de cada variable.

Binarización de producciones (conversión a Forma Normal de Chomsky)

Una vez depurada la gramática, toda producción con más de dos símbolos debe descomponerse en una cadena de producciones binarias equivalentes. El algoritmo aplica el mismo método utilizado en el proceso manual: se toma el primer símbolo de la producción y se empareja con una nueva variable auxiliar que representa el resto de la cadena; si ese resto todavía tiene más de dos símbolos, se repite el mismo procedimiento sobre él, de manera recursiva, creando una nueva variable auxiliar en cada nivel.

Las variables auxiliares se numeran de forma secuencial (X1, X2, X3…) en el mismo orden en que se necesitan, evitando cualquier nombre ya existente en la gramática. Cabe señalar una decisión de diseño particular de este proyecto: no se ejecuta un paso separado de sustitución de terminales antes de la binarización, ya que la forma de trabajo adoptada admite directamente combinaciones terminal-variable o variable-terminal dentro de una misma producción binaria (por ejemplo, B1, con B variable y 1 terminal), sin necesidad de aislar el terminal en una variable auxiliar aparte.

Validación de la Forma Normal de Chomsky

Como verificación final, el sistema recorre la totalidad de las producciones de la gramática resultante y confirma que cada una tenga, como máximo, dos símbolos, sin importar la combinación de terminales y variables que la compongan.

6. IMPLEMENTACIÓN

Lenguaje y tecnologías utilizadas

El motor de depuración y conversión se implementó en Python 3, por la claridad que ofrece su sintaxis para expresar algoritmos de teoría de conjuntos y manipulación de listas operaciones centrales en este proyecto, y por contar con soporte nativo para el despliegue como función serverless en la plataforma elegida. Sobre Python se utilizó el microframework Flask, encargado de exponer la lógica de depuración como una API HTTP.

La interfaz de usuario se desarrolló con React, utilizando Tailwind CSS para el diseño visual y Vite como herramienta de construcción. Todo el aplicativo tanto el frontend como el backend se despliega de forma conjunta en Vercel: la interfaz como sitio estático y la API de

Python como función serverless dentro de la misma plataforma, lo que permite que el estudiante acceda al sistema únicamente desde el navegador, sin instalación local.

Estructuras de datos

La estructura de datos central del sistema es la clase Gramatica, que representa formalmente la tupla G = (V, T, S, P):

- variables: conjunto (set) de variables o no terminales.
- terminales: conjunto (set) de símbolos terminales.
- inicial: cadena de texto con el símbolo inicial.
- producciones: diccionario (dict) donde cada clave es una variable y cada valor es una lista de producciones asociadas a ella.

La decisión de diseño más relevante en esta estructura es la forma en que se representa cada producción individual: no como una cadena de texto concatenada, sino como una tupla de símbolos (por ejemplo, ('A', 'A', 'B') en lugar de "AAB"). Esta decisión resultó necesaria porque, en las primeras versiones del sistema, las producciones se representaban como texto plano y se recorrían carácter por carácter; esto funcionaba correctamente mientras todas las variables tenían nombres de un solo carácter, pero dejó de ser válido en cuanto el proceso de conversión a Forma Normal de Chomsky comenzó a introducir variables auxiliares de más de un carácter (X1, X2, X10…), ya que una producción como "AX1" se interpreta incorrectamente como los símbolos A, X, 1 en lugar de A, X1. Adoptar la tupla de símbolos como unidad mínima de representación eliminó este problema de raíz.

Para el transporte de información entre el servidor y la interfaz, cada Gramatica se serializa a un objeto JSON equivalente, donde las tuplas de producciones se representan como listas de listas (y la producción nula, como una lista vacía), de modo que el frontend en JavaScript pueda reconstruir y mostrar la gramática sin ambigüedad.

El historial de transformaciones se apoya en dos estructuras adicionales: la clase Historial, que contiene una lista ordenada de pasos, y la clase PasoHistorial, que representa una transformación individual con la gramática antes y después del paso, los elementos identificados y las producciones eliminadas o agregadas.

Organización del código

Backend. El código Python se organiza en módulos según su responsabilidad:

- **gramatica.py:** define la clase Gramatica y sus operaciones básicas (copiar, agregar o eliminar una producción, eliminar una variable por completo).
- **utils.py:** funciones de apoyo para formatear y convertir producciones entre su representación interna y su forma de texto legible.
- **validador.py:** valida la estructura inicial de una gramática antes de permitir cualquier transformación.
- **depuracion.py:** implementa las cuatro fases de depuración —variables inútiles, variables inalcanzables, producciones nulas y producciones unitarias— junto con la función orquestadora depurar_gramatica().
- **chomsky.py:** implementa la binarización de producciones y la validación final de la Forma Normal de Chomsky.
- **generador.py:** construye gramáticas aleatorias para el modo de práctica.

- **historial.py:** contiene las clases Historial y PasoHistorial.
- **adaptador.py:** convierte entre los objetos internos de Python (Gramatica, PasoHistorial) y las estructuras JSON que consume el frontend.
- **index.py:** define la aplicación Flask y expone cada fase del proceso como un endpoint HTTP independiente (/api/validar, /api/inutiles, /api/inalcanzables, /api/nulas, /api/unitarias, /api/chomsky, /api/completo, /api/generar).

Frontend. La interfaz sigue una organización por componentes, coordinados por un componente principal (App.jsx) que mantiene el estado de la gramática actual, el historial acumulado y el número de estado Σ vigente:

- **GrammarForm:** formulario de registro de la gramática y generación de ejercicios de práctica.
- **GrammarPanel:** presenta la gramática actual con su notación formal completa.
- Toolbar: expone cada fase del proceso como una acción independiente, además del modo automático.
- **HistorialTimeline:** presenta cada paso registrado, incluyendo la gramática antes y después, y los elementos identificados, eliminados y agregados.
- **ChomskyStepView:** vista especializada para el proceso de binarización, que reproduce la notación manual de círculos para las producciones que quedan dentro de cada variable y chulos para las nuevas variables auxiliares creadas.
- **gramaticaUtils.js:** concentra el parseo de texto ingresado por el usuario y las funciones de comunicación con la API.

Métodos principales

Entre las funciones que concentran la lógica central del sistema se destacan:

- obtener_variables_generadoras() y obtener_variables_anulables(): calculan, mediante punto fijo, qué variables son generadoras y qué variables son anulables, respectivamente.
- eliminar_producciones_nulas(): resuelve la anulabilidad de la gramática variable por variable, detectando y rompiendo ciclos de dependencia que emergen a mitad de proceso.
- eliminar_producciones_unitarias(): resuelve las producciones unitarias variable de origen por variable de origen, con actualización automática por dependencia cuando una variable de destino cambia en un paso posterior.
- binarizar_producciones(): descompone recursivamente las producciones de más de dos símbolos, generando y enumerando las variables auxiliares necesarias.
- depurar_gramatica() y convertir_a_fnc(): orquestan la ejecución completa de las fases anteriores para el modo automático del sistema.

Comunicación cliente-servidor

La interfaz y el motor de depuración se comunican mediante peticiones HTTP en formato JSON. Cada acción disponible en la interfaz validar, ejecutar una fase individual, ejecutar el proceso completo o generar un ejercicio corresponde a una petición independiente hacia el backend, el cual responde con la gramática resultante y el conjunto de pasos generados durante esa operación. El número de estado Σ de cada paso se calcula del lado del cliente, comparando el estado de la gramática antes y después de cada transformación recibida, de modo que el contador solo avanza cuando una fase produce un cambio real sobre la gramática.

7. PRUEBAS

Las pruebas presentadas a continuación corresponden a casos reales trabajados durante el desarrollo del proyecto, varios de ellos formulados manualmente por el equipo para verificar que el sistema replicara exactamente el procedimiento enseñado en clase. En cada una se indica la gramática inicial, el proceso ejecutado, los resultados intermedios relevantes, el resultado esperado (calculado a mano) y el resultado obtenido por el sistema.

Prueba 1 Validación inicial de la gramática

Gramática inicial:

$G = (\{A, B\}, X, \{1\}, \Sigma)$

$A \rightarrow 1B$

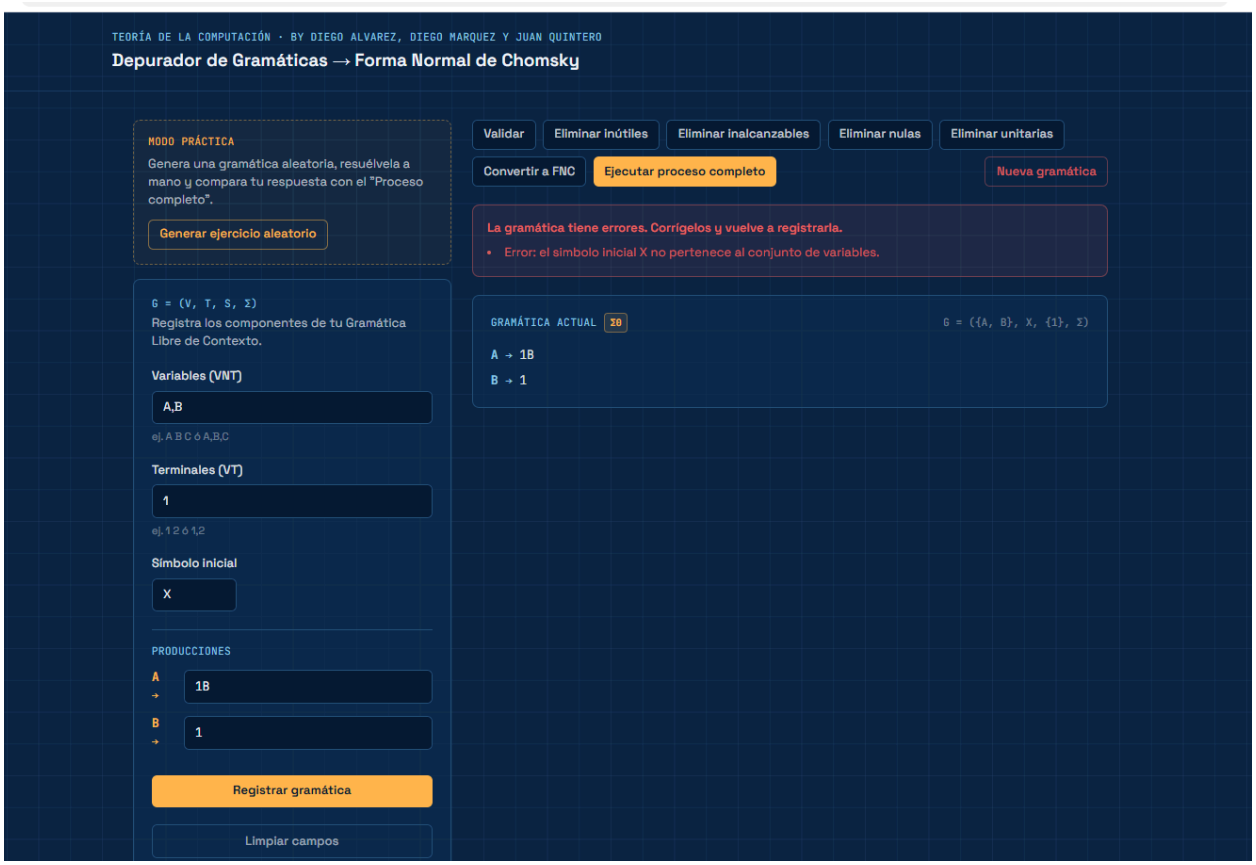
$B \rightarrow 1$

Proceso realizado: Registro de la gramática y ejecución de la validación estructural.

Resultado esperado: El sistema debe rechazar la gramática, ya que el símbolo inicial declarado (X) no pertenece al conjunto de variables.

Resultado obtenido: El sistema bloqueó el proceso y mostró el mensaje *"Error: el símbolo inicial X no pertenece al conjunto de variables"*, impidiendo avanzar a las fases de depuración hasta corregir el error, tal como lo exige el flujo definido en el diagrama de actividades.

Figura 4. Validación inicial de la gramática



Nota. Capture tomado del depuradorfnc

Prueba 2 Eliminación de variables inútiles (símbolos no declarados)

Gramática inicial:

G = ({A, B, C, D, E, F}, A, {1, 2, 3}, Σ)

A → BB1 | GF2 | 3

B → 2E | 2DE | 2

C → E

D → 12C | B3C1 | 2

E → CC2 | DCDC2 | DCC2 | CDC2 | 1

F → (sin producciones)

Proceso realizado: Fase de eliminación de variables inútiles.

Resultado esperado: El sistema debe reconocer dos casos distintos de utilidad: la variable F, declarada sin ninguna producción propia, y el símbolo G, usado dentro de una producción de A sin haber sido declarado. Ambos deben eliminarse, junto con cualquier producción que los contenga.

Resultado obtenido:

Después

G = ({A, B, C, D, E}, A, {1, 2, 3}, Σ)

A → BB1 / 3

B → 2E / 2DE / 2

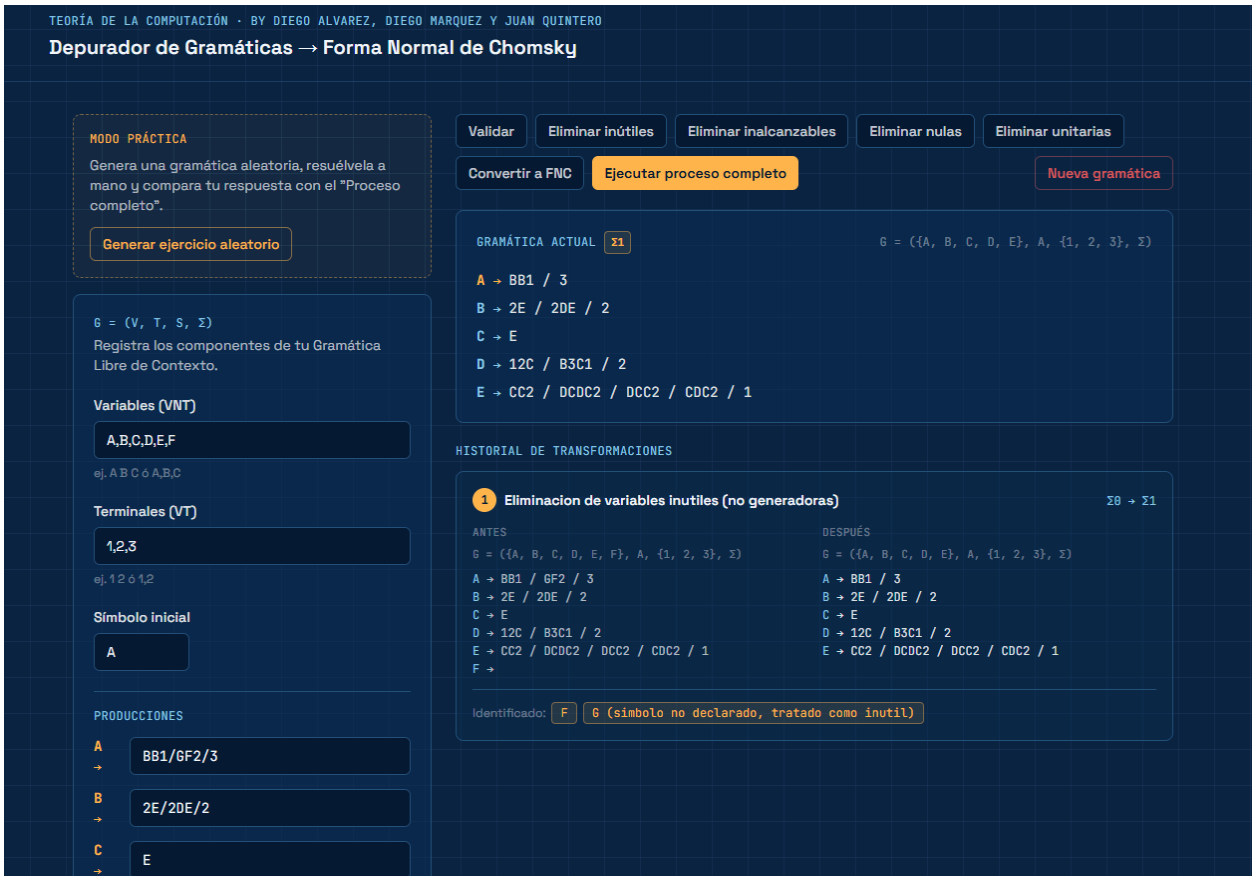
C → E

D → 12C / B3C1 / 2

E → CC2 / DCDC2 / DCC2 / CDC2 / 1

La producción A → GF2 desapareció por completo (contenía a G), y tanto F como G fueron correctamente identificados y eliminados, coincidiendo con el resultado esperado.

Figura 5. Eliminación de variables inútiles (símbolos no declarados)



Nota. Capture tomado del depuradorfnc

Prueba 3 Eliminación de producciones nulas con ciclo emergente a mitad de proceso

Gramática inicial (tras inútiles e inalcanzables):

G = ({A, B, C}, A, {1, 2, 3}, Σ)

A → BB | 2 | C | 213 | 3

$B \rightarrow A \mid B1 \mid 1 \mid \lambda \mid 3$

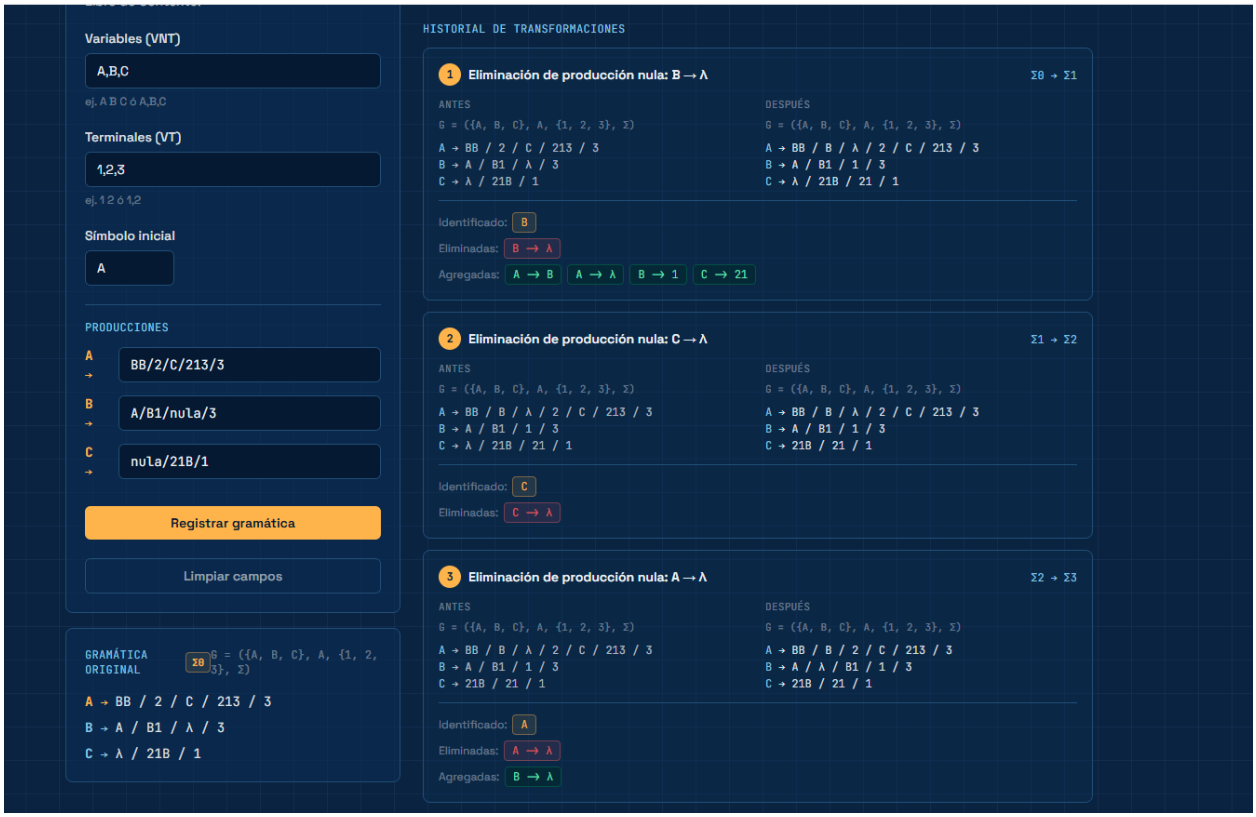
$C \rightarrow \lambda \mid 21B \mid 1$

Proceso realizado: Eliminación de producciones nulas, una variable a la vez.

Resultado esperado: Al eliminar la nula de B, la producción BB de A se reduce a B (quitando ambas ocurrencias de B), lo que introduce por primera vez una producción unitaria $A \rightarrow B$. Como B a su vez tiene la producción A, se forma un ciclo de dependencia mutua ($A \rightarrow B$, $B \rightarrow A$) que no existía en la gramática original y que solo aparece a mitad del proceso. El sistema debe detectarlo y romperlo automáticamente ejecutando una pasada de eliminación de unitarias antes de continuar, sin dejar ninguna producción nula residual al finalizar.

Resultado obtenido: El sistema procesó $B \rightarrow \lambda$, luego $C \rightarrow \lambda$, detectó que A necesitaba resolverse por segunda vez (señal de ciclo), ejecutó automáticamente la eliminación de unitarias correspondiente, y retomó el barrido de nulas hasta completarlo. La gramática final no conservó ninguna producción nula residual en variables distintas al símbolo inicial, confirmando que el mecanismo de detección de ciclos emergentes funciona correctamente.

Figura 6. Eliminación de producciones nulas con ciclo emergente a mitad de proceso



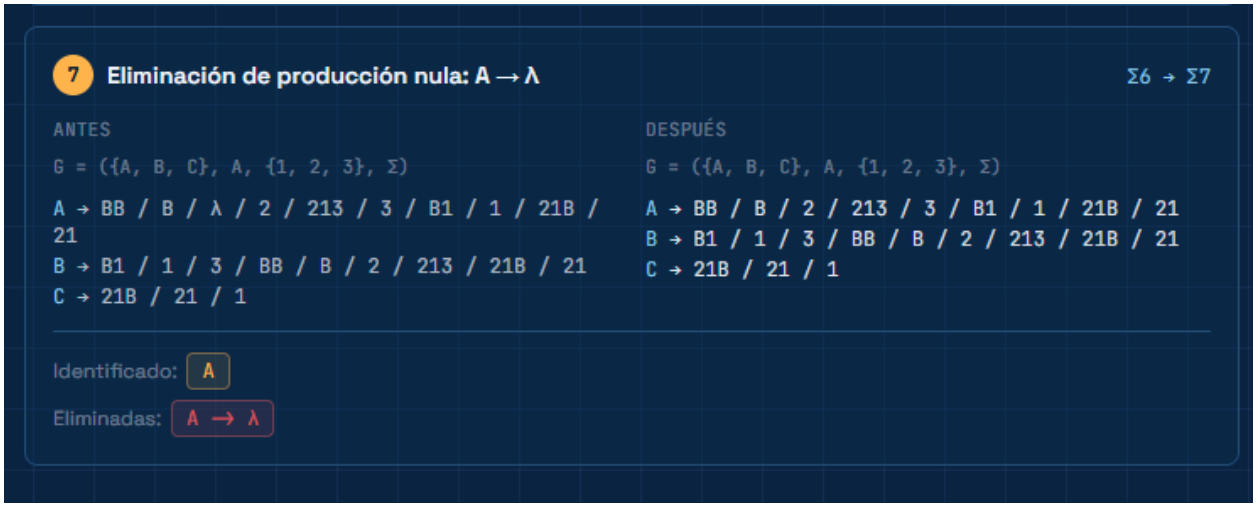
Nota. Capture tomado del depuradorfnc

Figura 7. Eliminación de producciones nulas con ciclo emergente a mitad de proceso



Nota. Capture tomado del depuradorfnc

Figura 8. Eliminación de producciones nulas con ciclo emergente a mitad de proceso



Nota. Capture tomado del depuradorfnc

(Este caso corresponde a un error real identificado durante el desarrollo, en una versión anterior del sistema, donde la variable regeneraba una producción nula sin ser reprocesada. Ver sección 8, Resultados, para más detalle.)

Prueba 4 Eliminación de producciones unitarias en cadena (un nivel a la vez)

Gramática inicial:

G = ({A, B, D, E}, A, {1, 2}, Σ)

A → EE | 1

B → D | 1

D → E | EA | 2

E → E2 | B1 | 22

Proceso realizado: Eliminación de producciones unitarias.

Resultado esperado: En el turno de B, la sustitución de B → D debe usar únicamente las producciones que D tiene en ese momento, excluyendo la unitaria propia de D (D → E), obteniendo B → EA | 2 | 1. Cuando más adelante D resuelva su propia unitaria hacia E, B debe actualizarse automáticamente para heredar ese nuevo contenido, sin que el sistema mezcle ambos niveles en un solo paso.

Resultado obtenido:

Después

G = ({A, B, D, E}, A, {1, 2}, Σ)

A → EE / 1

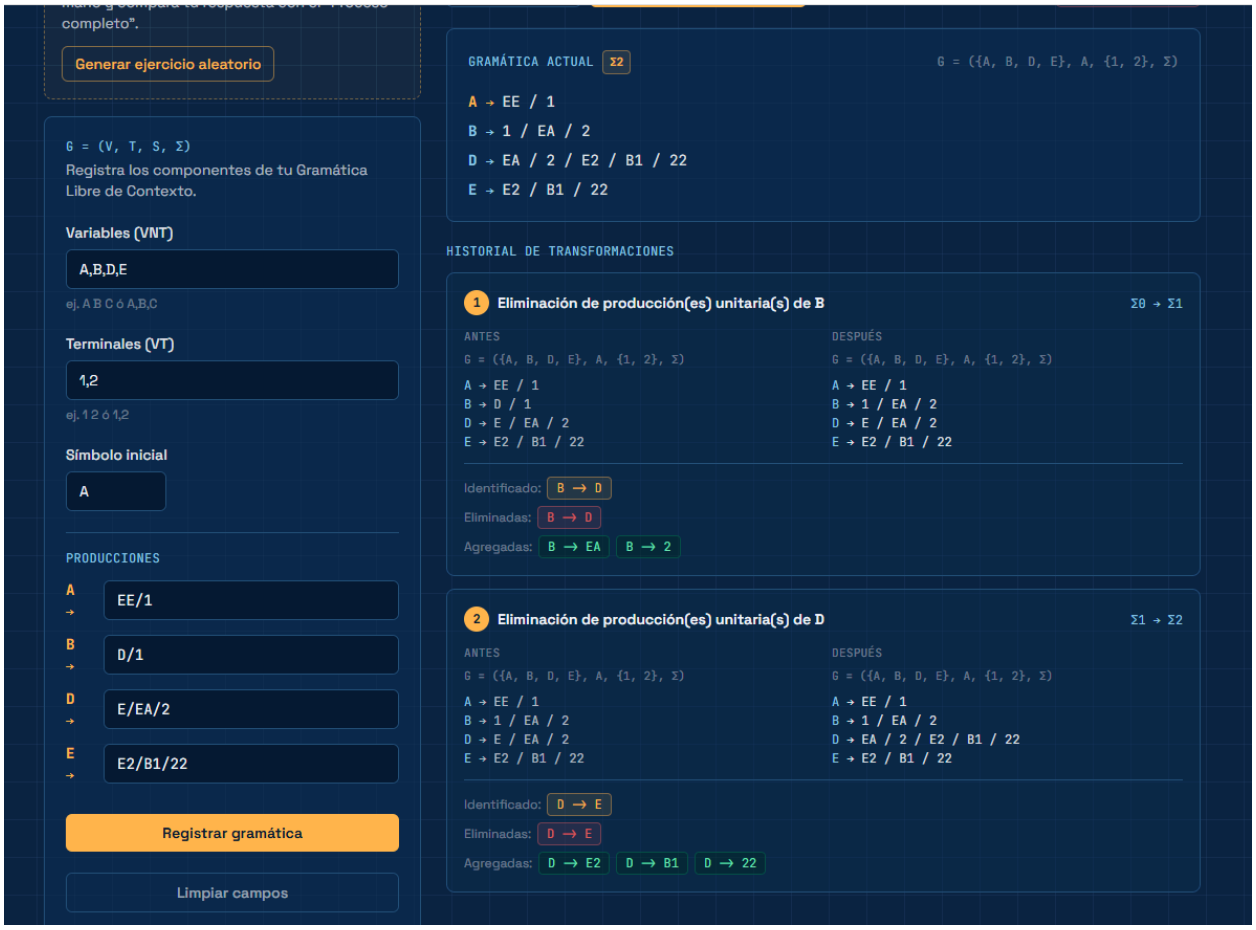
B → 1 / EA / 2

D → EA / 2 / E2 / B1 / 22

E → E2 / B1 / 22

El sistema respetó la sustitución de un solo nivel en el turno de B, y posteriormente propagó el cambio de D hacia B en un paso independiente, coincidiendo exactamente con el resultado esperado.

Figura 9. Eliminación de producciones unitarias en cadena (un nivel a la vez)



Nota. Capture tomado del depuradorfnc

Prueba 5 Conversión a Forma Normal de Chomsky

Gramática inicial (depurada):

G = ({A, B}, A, {0, 1, 2}, Σ)

A → ABAB | ABB | AAB | 1 | 0

B → ABA | AAB | 1 | 2

Proceso realizado: Binarización de las producciones de longitud mayor a dos símbolos.

Resultado esperado:

Gramática actual

Σ1

G = ({A, B, X1, X2, X3, X4, X5, X6}, A, {0, 1, 2}, Σ)

A →

AX1 / AX3 / AX4 / 1 / 0

B →

AX5 / AX6 / 1 / 2

X1 →

BX2

X2 →

AB

X3 →

BB

X4 →

AB

X5 →

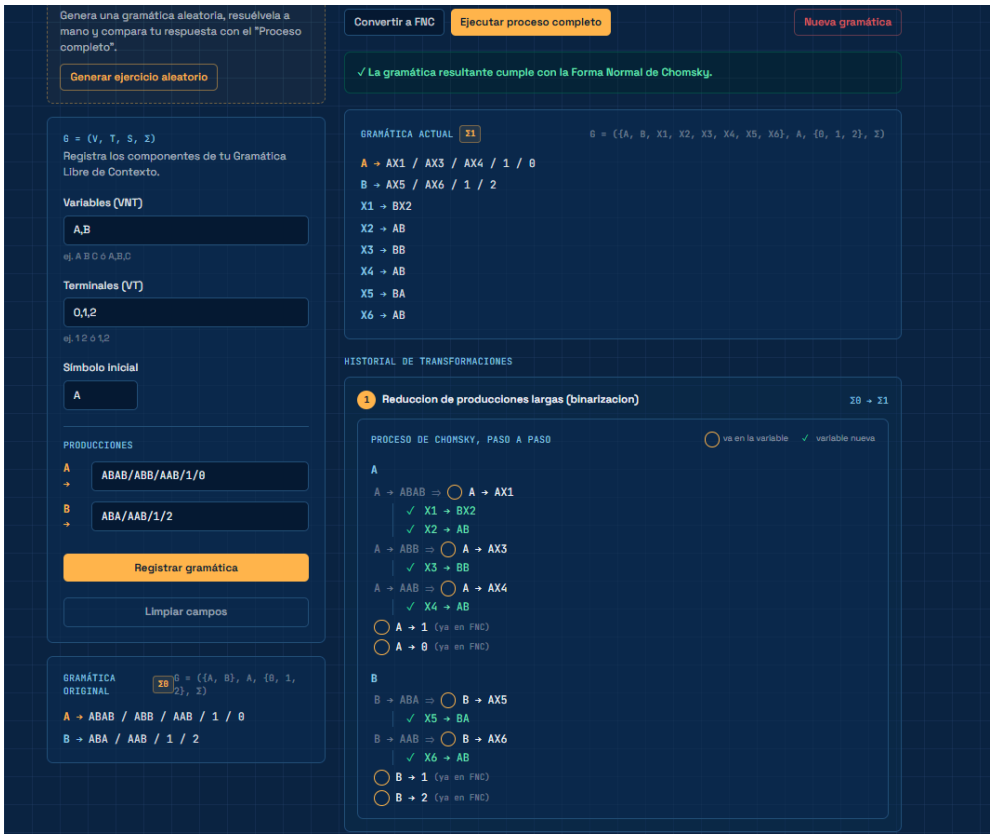
BA

X6 →

AB

Resultado obtenido: El sistema generó exactamente la misma gramática, con la misma numeración de variables auxiliares (X1 a X6) y la misma estructura de descomposición recursiva, coincidiendo símbolo por símbolo con el cálculo manual.

Figura 10. Conversión a Forma Normal de Chomsky



Nota. Capture tomado del depuradorfnc

Prueba 6 Integración completa

Gramática inicial:

G = ({A, B, C, D, F}, A, {1, 2}, Σ)

A →

ABC / ABG / BB2 / 1 / 2

B →

D / 12 / BA / DDC

C →

ADA / 1 / 2 / λ

D →

AF2 / BAD / CAD / 1

F →

(sin producciones)

Proceso realizado: Ejecución completa del proceso de depuración (modo manual, fase por fase), como prueba de integración de todos los algoritmos descritos en la sección 5.

Resultado esperado: Eliminación de F (inútil) y G (No declarado dentro de la Gramática), ninguna variable inalcanzable, resolución completa de la producción nula C y unitaria B, sin ninguna producción nula residual y unitaria al finalizar. Luego de la depuración se pasa a formalizar a FNC y se hace el proceso de binarización encerrando en círculo las nuevas producciones y con chulo las nuevas variables de chomsky que irán dentro del nuevo sigma con su equivalente ya formalizado.

G = ({A, B, C, D, X1, X2, X3, X4, X5, X6, X7, X8}, A, {1, 2}, Σ)

A →

AX1 / AB / BX2 / 1 / 2

B →

12 / BA / DX3 / DD / BX4 / CX5 / AD / 1

C →

AX6 / 1 / 2

D →

BX7 / CX8 / AD / 1

X1 →

BC

X2 →

B2

X3 →

DC

X4 →

AD

X5 →

AD

X6 →

DA

X7 →

AD

X8 →

AD

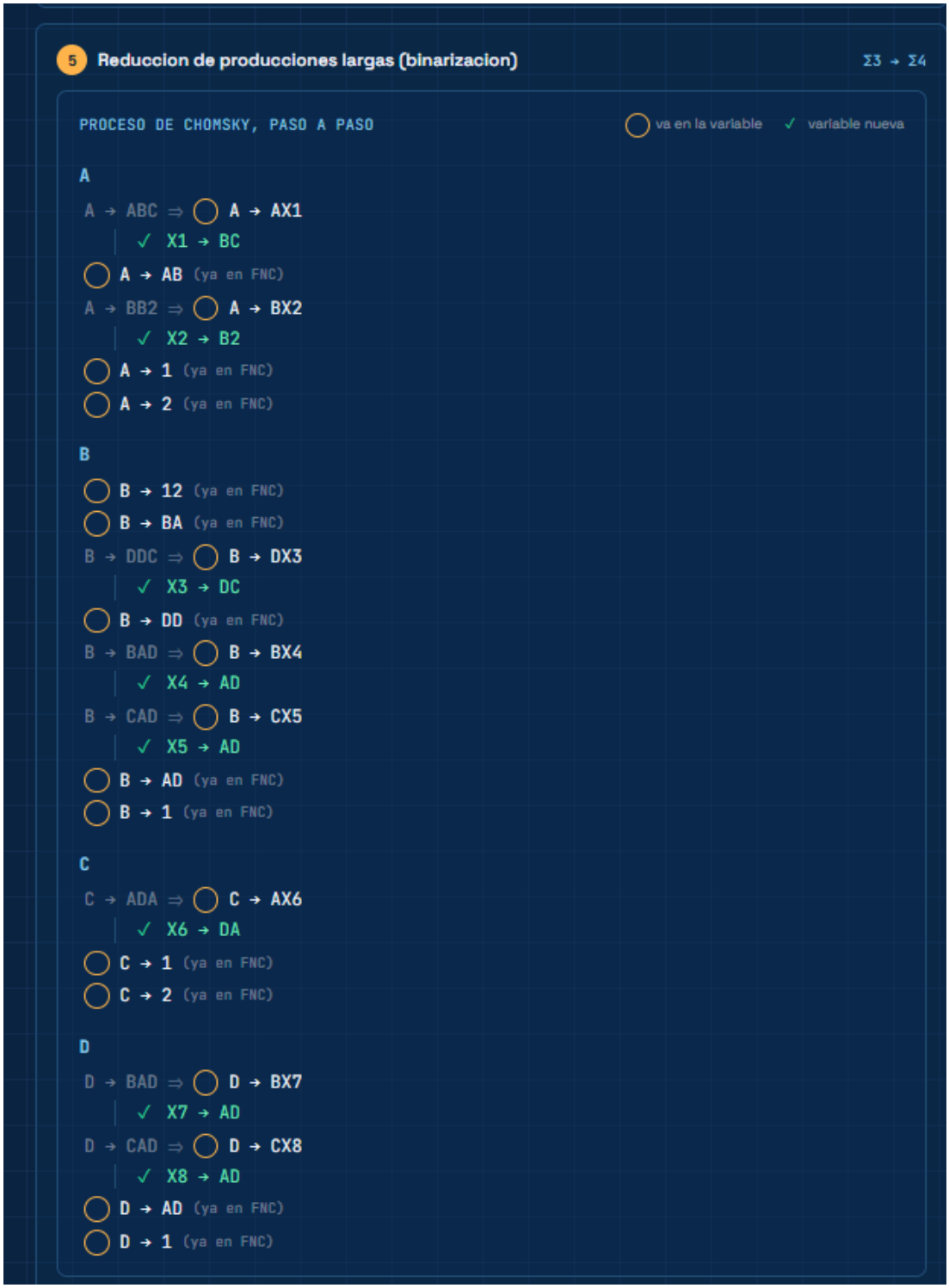
Resultado obtenido: El proceso se resolvió en 5 pasos, sin producciones nulas residuales en la gramática final, tal como se esperaba matemáticamente. El detalle completo, paso a paso, se encuentra documentado en este documento y se puede visualizar mejor en la figura 10 y 12 de la prueba 5 en la tabla de figuras.

Figura 11. Integración completa



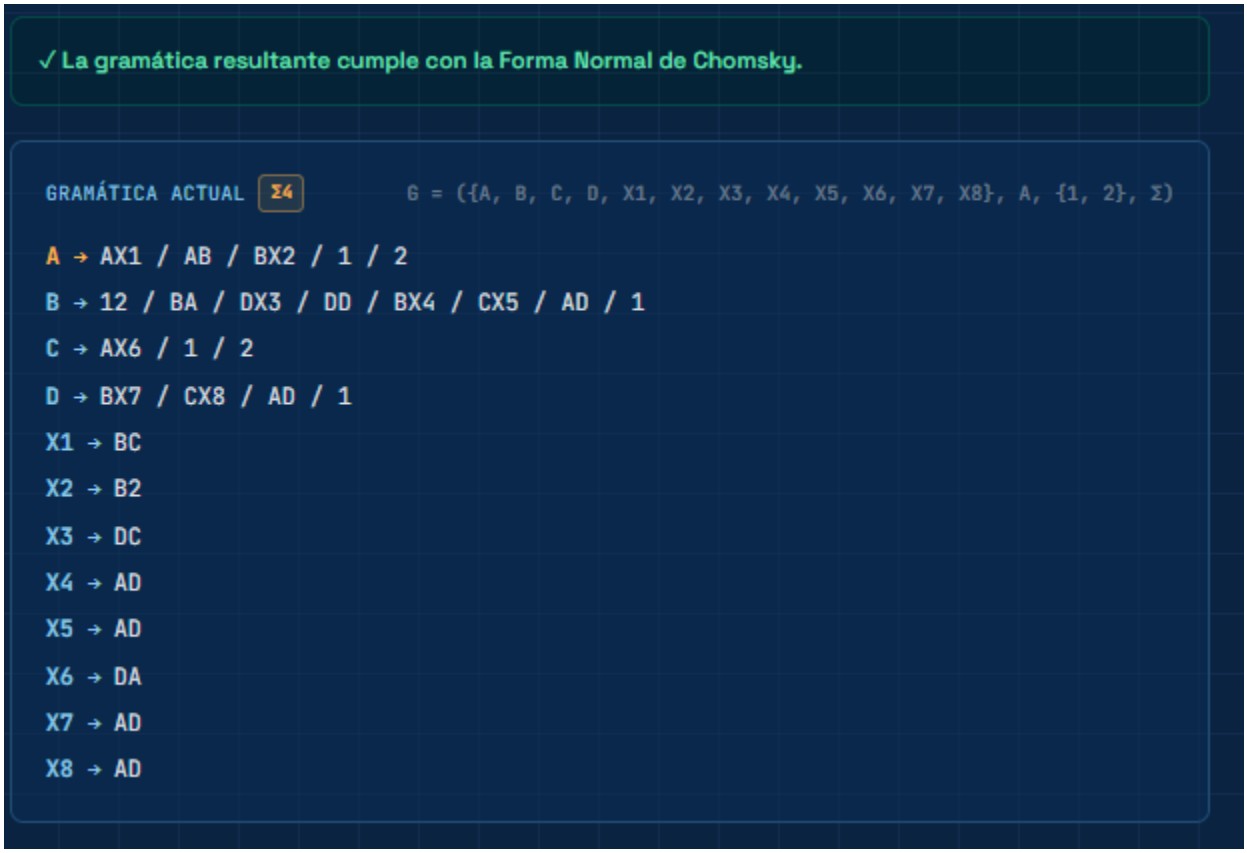
Nota. Capture tomado del depuradorfnc

Figura 12. Integración completa



Nota. Capture tomado del depuradorfnc

Figura 13. Integración completa



Nota. Capture tomado del depuradorfnc

Resumen de resultados de pruebas

Tabla 3

Resultados Pruebas

PRUEBA	ASPECTO VERIFICADO	RESULTADO
1	Validación estructural de la gramática	Correcto
2	Eliminación de variables inútiles (declaradas y no	Correcto

	declaradas)	
3	Detección de ciclos emergentes en producciones nulas	Correcto
4	Sustitución de un solo nivel en producciones unitarias	Correcto
5	Binarización y numeración de variables auxiliares	Correcto
6	Integración completa	Correcto

8. RESULTADOS

Resultados generales

El desarrollo del proyecto permitió obtener un aplicativo web completamente funcional, capaz de recibir cualquier Gramática Libre de Contexto ingresada por el usuario, validarla, depurarla en sus cuatro fases (variables inútiles, variables inalcanzables, producciones nulas y producciones unitarias) y convertirla a su Forma Normal de Chomsky, tanto en modo manual fase por fase como en modo automático. El sistema quedó desplegado como una aplicación accesible desde cualquier navegador, sin requerir instalación local, con un backend en Python que concentra toda la lógica formal del proceso y una interfaz en React que reproduce visualmente la misma notación y el mismo orden de trabajo empleados en el desarrollo manual del tema.

Como se evidencia en las pruebas de la sección 7, el sistema procesó correctamente casos que van desde gramáticas simples de dos variables hasta gramáticas de siete variables con

múltiples niveles de dependencia entre producciones nulas y unitarias, sin comprometer en ningún caso la equivalencia entre la gramática original y la gramática resultante.

Robustez del proceso frente a casos límite

Uno de los resultados más significativos del proyecto no se obtuvo en el diseño inicial de los algoritmos, sino en el proceso posterior de verificación exhaustiva, en el que se pusieron a prueba deliberadamente casos límite que no siempre son evidentes al momento de programar. Este ejercicio permitió identificar y corregir dos situaciones concretas que, de no haberse detectado, habrían comprometido la validez del resultado final:

- **Ciclos de dependencia que emergen a mitad del proceso**. Inicialmente, el sistema solo verificaba la existencia de un ciclo entre producciones nulas y unitarias una vez, al inicio del proceso de depuración. Las pruebas evidenciaron que un ciclo de este tipo puede no existir en la gramática original y aparecer únicamente como efecto colateral de una eliminación anterior por ejemplo, cuando una producción como BB se reduce a B al eliminar una nula, generando una unitaria que antes no existía. Esto llevó a rediseñar el algoritmo para que la detección de ciclos sea continua durante todo el proceso, y no una verificación única al comienzo.
- **Sustitución prematura en cadenas de producciones unitarias**. Una primera versión del algoritmo de eliminación de unitarias resolvía el cierre transitivo completo de una variable en un solo paso, lo que provocaba que una variable heredara contenido de niveles de dependencia que aún no le correspondían. La corrección de este comportamiento llevó al diseño del mecanismo de actualización por dependencia descrito

en la sección 5, que resuelve cada nivel por separado y propaga los cambios automáticamente cuando corresponde.

La corrección de estos dos casos, cada uno descubierto mediante pruebas específicamente diseñadas para exigir al sistema más allá de los ejemplos convencionales, es en sí misma uno de los resultados más valiosos del proyecto: evidencia que la implementación no se limitó a reproducir el algoritmo en su forma más simple, sino que fue sometida a un proceso de verificación riguroso hasta garantizar su corrección frente a los casos que, precisamente, son los que con mayor frecuencia generan errores en la resolución manual.

Funcionalidades adicionales logradas

Más allá de los requerimientos formales del proceso de depuración y conversión, el desarrollo permitió incorporar funcionalidades orientadas a fortalecer el componente formativo del aplicativo:

- **Un generador de gramáticas aleatorias**, que produce ejercicios de práctica con casos representativos de cada fase (variables sin producciones, símbolos no declarados, producciones nulas y unitarias), garantizando además que el resultado nunca degenere en una gramática vacía.
- Una **vista especializada para el proceso de Chomsky**, que diferencia visualmente, mediante la misma notación de círculos y chulos empleada en el desarrollo manual, las producciones que quedan dentro de cada variable de las nuevas variables auxiliares creadas.

- Un **sistema de numeración progresiva de estados (Σ)**, que solo avanza cuando una fase produce un cambio real sobre la gramática, replicando fielmente la forma en que se lleva el control del proceso en el desarrollo manual.

Limitaciones identificadas

Durante el desarrollo se identificaron también algunas limitaciones que quedan como oportunidades de mejora. La más relevante es que la forma normal implementada acepta cualquier combinación de dos símbolos en una producción binaria (terminal-terminal, terminal-variable, variable-terminal o variable-variable), en lugar de exigir la definición clásica más estricta de la Forma Normal de Chomsky, que solo admite producciones $A \rightarrow BC$ o $A \rightarrow a$. Esta decisión se adoptó deliberadamente para ajustarse a la forma de trabajo indicada por la docente de la asignatura, pero representa una diferencia respecto a la definición formal presentada en buena parte de la literatura de referencia, y debe tenerse presente si el aplicativo llegara a usarse en un contexto donde se exija la definición estricta.

También queda como mejora para un futuro emplear que en la depuración el sistema puede detectar y saber que pasos son mejores e ir combinando nulas y unitarias para sacar el menor número de pasos posibles.

9. CONCLUSIONES

El desarrollo de este microproyecto permitió confirmar algo que no siempre resulta evidente al estudiar la teoría de lenguajes formales desde el papel: comprender los conceptos de variable anulable, producción unitaria o Forma Normal de Chomsky es solo el primer paso, pero convertir esa comprensión en un algoritmo que se ejecute correctamente, sobre cualquier gramática y en cualquier orden posible de dependencias, exige un nivel de rigor considerablemente mayor. Varias reglas que en un ejemplo sencillo parecen triviales como el hecho de que un ciclo entre producciones nulas y unitarias puede no existir en la gramática original y aparecer únicamente a mitad de proceso solo se hacen evidentes cuando se intenta programarlas de forma general, y no simplemente reproducirlas para un caso particular.

En ese sentido, el proceso de verificación resultó tan formativo como el de implementación. Los dos casos límite corregidos durante el desarrollo la detección continua de ciclos emergentes y la sustitución de un solo nivel en las cadenas de producciones unitarias no se identificaron leyendo la teoría una vez más, sino sometiendo el sistema a ejercicios deliberadamente exigentes y comparando, paso a paso, su comportamiento contra el procedimiento manual enseñado en clase. Esa disciplina de contrastar cada transformación intermedia, y no solo el resultado final, fue lo que permitió detectar errores que de otra forma habrían pasado desapercibidos.

El proyecto también dejó una lección sobre la naturaleza misma de estos algoritmos: su dificultad no radica en la complejidad matemática de cada regla individual, sino en la interdependencia entre variables. Una sola eliminación de una variable inútil, de una producción nula puede tener efectos indirectos sobre partes de la gramática que ya se consideraban resueltas,

y sostener esa trazabilidad de forma consistente es, en la práctica, la verdadera dificultad del proceso, tanto al resolverlo a mano como al programarlo.

Finalmente, la construcción de este aplicativo confirma el valor de herramientas que no reemplazan el razonamiento algorítmico que exige la materia, sino que lo acompañan: al reproducir la misma notación, el mismo orden de fases y el mismo tratamiento de los casos especiales que se trabajan manualmente, el sistema le permite a cualquier estudiante contrastar su propio proceso de resolución contra un resultado verificado, identificar con precisión en qué paso se desvió si lo hizo, y consolidar con mayor seguridad un tipo de razonamiento procedimental que seguirá siendo relevante en áreas como el diseño de compiladores y el procesamiento de lenguajes formales.

10. REFERENCIAS BIBLIOGRÁFICAS

Chomsky, N. (1959). On certain formal properties of grammars. *Information and Control*, *2*(2), 137–167. https://doi.org/10.1016/S0019-9958(59)90362-6

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.ª ed.). Pearson/Addison-Wesley.

Meta Open Source. (s. f.). *React – The library for web and native user interfaces*. React. Recuperado el 29 de septiembre de 2026, de https://react.dev

Pallets Projects. (s. f.). *Flask documentation*. Recuperado el 29 de septiembre de 2026, de https://flask.palletsprojects.com

Sipser, M. (2013). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning.

Tailwind Labs. (s. f.). *Tailwind CSS documentation*. Recuperado el 29 de septiembre de 2026, de https://tailwindcss.com/docs

Vercel Inc. (s. f.). *Vercel documentation*. Recuperado el 29 de septiembre de 2026, de https://vercel.com/docs

Younger, D. H. (1967). Recognition and parsing of context-free languages in time n^3. *Information and Control*, *10*(2), 189–208. https://doi.org/10.1016/S0019-9958(67)80007-X